

"The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system."

[Wikipedia: Singleton Pattern](#)

Complete the Singleton class in your editor which contains the following components:

1. A private Singleton non parameterized constructor.
2. A public String instance variable named `str`.
3. Write a static method named `getSingletonInstance` that returns the single instance of the Singleton class.

Once submitted, our hidden Solution class will check your code by taking a String as input and then using your Singleton class to print a line.

Input Format

You will not be handling any input in this challenge.

Output Format

You will not be producing any output in this challenge.

Sample Input

hello world

Sample Output

Hello I am a singleton! Let me say hello world to you

```
33
34 //verify that the constructor is private
35 Class c=s1.getClass();
36 Constructor[] allConstructors = c.getDeclaredConstructors();
37 assert allConstructors.Length==1;
38 for (Constructor ctor : allConstructors)
39
```

Lines: 23 Cols: 2

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

```
1 hello world
```

Your Output (stdout)

```
1 Hello I am a singleton! Let me say hello world to you
```

Expected Output

Download

```
1 Hello I am a singleton! Let me say hello world to you
```

You are given the partially completed code in the editor. Please complete the FoodFactory class.

cake

```
The factory returned class Cake
Someone ordered a Dessert!
```

pizza

```
The factory returned class Pizza
Someone ordered Fast Food!
```

Line: 23 Col: 10

 Upload Code as File☐ Test against custom input

Run Code

Submit Code

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

```
input(stdin)
```

Download

cake

Your Output (stdout)

```
1 The factory returned class Cake
2 Someone ordered a Dessert!
```

Expected Output

Download

```
1 The factory returned class Cake
2 Someone ordered a Dessert!
```

Problem

Submissions

Leaderboard

Discussions

Editorial

You are required to compute the power of a number by implementing a calculator. Create a class `MyCalculator` which consists of a single method `long power(int, int)`. This method takes two integers, `n` and `p`, as parameters and finds n^p . If either `n` or `p` is negative, then the method must throw an exception which says "**`n` or `p` should not be negative**". Also, if both `n` and `p` are zero, then the method must throw an exception which says "**`n` and `p` should not be zero.**"

For example, `-4` and `-5` would result in `java.lang.Exception: n or p should not be negative`.

Complete the function `power` in class `MyCalculator` and return the appropriate result after the power operation or an appropriate exception as detailed above.

Input Format

Each line of the input contains two integers, `n` and `p`. The locked stub code in the editor reads the input and sends the values to the method as parameters.

Constraints

- $-10 \leq n \leq 10$
- $-10 \leq p \leq 10$

Output Format

Each line of the output contains the result n^p , if both `n` and `p` are positive. If either `n` or `p` is negative, the output contains "`n` and `p` should be non-negative". If both `n` and `p` are zero, the output contains "`n` and `p` should not be zero.". This is printed by the locked stub code in the editor.

Sample Input 0

```
3 5
2 4
0 0
-1 -2
-1 3
```

Sample Output 0

```
243
16
java.lang.Exception: n and p should not be zero.
java.lang.Exception: n or p should not be negative.
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 3 5
2 2 4
3 0 0
4 -1 -2
5 -1 3
```

Download

Your Output (stdout)

```
1 243
2 16
3 java.lang.Exception: n and p should not be zero.
4 java.lang.Exception: n or p should not be negative.
```

Problem

Submissions

Leaderboard

Discussions

Editorial

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). $\{1, 2, 3\}$ is an example of a set, but $\{1, 2, 2\}$ is not a set. Today you will learn how to use sets in java by solving this problem.

You are given n pairs of strings. Two pairs (a, b) and (c, d) are identical if $a = c$ and $b = d$. That also implies (a, b) is not same as (b, a) . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings separated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i^{th} line, print number of unique pairs you have after taking i^{th} pair as input.

Sample Input

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Explanation

Peaks: 100%

Line: 24 Col: 10

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

```
1 5
2 john tom
3 john mary
4 john tom
5 mary anna
6 mary anna
```

Your Output (stdout)

```
1 1
2 2
3 2
```

Problem

Submissions

Leaderboard

Discussions

Editorial

When a subclass inherits from a superclass, it also inherits its methods; however, it can also override the superclass methods (as well as declare and implement new ones). Consider the following Sports class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}
```

Next, we create a Soccer class that inherits from the Sports class. We can override the getName method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the `@Override` annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task

Complete the code in your editor by writing an overridden `getNumberOfTeamMembers` method that prints the same statement as the superclass' `getNumberOfTeamMembers` method, except that it replaces `n` with **11** (the number of players on a Soccer team).

Output Format

When executed, your completed code should print the following:

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

30c1.getNumberOfTeamMembers();
31System.out.println(c2.getName());
32c2.getNumberOfTeamMembers();
33}
34}
35

Line: 18 Col: 1

Upload Code as FileTest against custom inputRun CodeSubmit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)
1Generic Sports
2Each team has n players in Generic Sports
3Soccer Class
4Each team has 11 players in Soccer Class

Expected Output
1Generic Sports
2Each team has n players in Generic Sports
3Soccer Class
4Each team has 11 players in Soccer Class

Download

32°C Mostly sunny

Search

11:37 AM 9/15/2024

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

Line: 18 Col: 1

☐ Test against custom input

Submit Code

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Your Output (stdout)

Download

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

Problem

Submissions

Leaderboard

Discussions

Editorial

A Java interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface `AdvancedArithmetic` which contains a method signature `int divisor_sum(int n)`. You need to write a class called `MyCalculator` which implements the interface.

`divisorSum` function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so `divisor_sum` should return 12. The value of `n` will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the `MyCalculator` class only. Your class shouldn't be public.

Sample Input

6

Sample Output

I implemented: AdvancedArithmetic
12

Explanation

Divisors of 6 are 1,2,3 and 6. 1+2+3+6=12.

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Line: 9 Col: 2

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Sample Test case 2

Input (stdin)

Download

1 6

Your Output (stdout)

1 I implemented: AdvancedArithmetic
2 12

Download

Expected Output

1 I implemented: AdvancedArithmetic
2 12

Problem

Submissions

Leaderboard

Discussions

Editorial

Java Iterator class can help you to iterate through every element in a collection. Here is a simple example:

```
import java.util.*;

public class Example{

    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        mylist.add("Hello");
        mylist.add("Java");
        mylist.add("4");
        Iterator it = mylist.iterator();
        while(it.hasNext()){
            Object element = it.next();
            System.out.println(String.valueOf(element));
        }
    }
}
```

In this problem you need to complete a method func. The method takes an ArrayList as input. In that ArrayList there is one or more integer numbers, then there is a special string "###", after that there are one or more other strings. A sample ArrayList may look like this:

```
element[0] => 42
element[1] => 10
element[2] => "###"
element[3] => "Hello"
element[4] => "Java"
```

You have to modify the func method by editing at most 2 lines so that the code only prints the elements after the special string "###". For the sample above the output will be:

```
Hello
Java
```

Note: The stdin doesn't contain the string "###", it is added in the main method.

To restore the original code in the editor, click the top left icon on the editor and create a new buffer.

Lines: 7, Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Download

Input (stdin)

1 2 2
2 42
3 10
4 hello
5 java

Your Output (stdout)

1 hello
2 java

Expected Output

Download

Problem

Submissions

Leaderboard

Discussions

Note: In this problem you **must NOT** generate any output on your own. Any such solution will be considered as being against the rules and its author will be disqualified. The output of your solution must be generated by the uneditable code provided for you in the solution template.

An important concept in Object-Oriented Programming is the *open/closed principle*, which means writing code that is open to extension but closed to modification. In other words, new functionality should be added by writing an extension for the existing code rather than modifying it and potentially breaking other code that uses it. This challenge simulates a real-life problem where the open/closed principle can and should be applied.

A Tree class implementing a rooted tree is provided in the editor. It has the following publicly available methods:

- `getValue()`: Returns the value stored in the node.
- `getColor()`: Returns the color of the node.
- `getDepth()`: Returns the *depth* of the node. Recall that the depth of a node is the number of edges between the node and the tree's root, so the tree's root has depth 0 and each descendant node's depth is equal to the depth of its parent node + 1.

In this challenge, we treat the internal implementation of the tree as being closed to modification, so we cannot directly modify it; however, as with real-world situations, the implementation is written in such a way that it allows external classes to extend and build upon its functionality. More specifically, it allows objects of the `TreeNode` class ([a Visitor Design Pattern](#)) to visit the tree and traverse the tree structure via the `accept` method.

There are two parts to this challenge.

Part I: Implement Three Different Visitors

Each class has three methods you must write implementations for:

1. `getResult()`: Return an integer denoting the *result*, which is different for each class:
 - The `SumInLeavesVisitor` implementation must return the sum of the values in the tree's leaves only.
 - The `ProductRedNodesVisitor` implementation must return the product of values stored in all red nodes, including leaves, computed modulo $10^9 + 7$. Note that the product of zero values is equal to 1.
 - The `FancyVisitor` implementation must return the absolute difference between the sum of values stored in the tree's non-leaf nodes at even depth and the sum of values stored in the tree's green leaf nodes. Recall that zero is an even number.
2. `visitNode(TreeNode node)`: Implement the logic responsible for visiting the tree's non-leaf nodes such that the `getResult` method returns the correct *result* for the implementing class' visitor.

```
211 int res3 = vis3.getResult();
212
213 System.out.println(res1);
214 System.out.println(res2);
215 System.out.println(res3);
216 }
217
```

Line: 217 Col: 2

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

```
1 5
2 4 7 2 5 12
3 0 1 0 0 1
4 1 2
5 1 3
6 3 4
7 3 5
```

Your Output (stdout)

```
1 24
2 40
```

Problem

Submissions

Leaderboard

Discussions

Editorial

Exception handling is the process of responding to the occurrence, during computation, of exceptions — anomalous or exceptional conditions requiring special processing — often changing the normal flow of program execution. (Wikipedia)

Java has built-in mechanism to handle exceptions. Using the try statement we can test a block of code for errors. The catch block contains the code that says what to do if exception occurs.

This problem will test your knowledge on try-catch block.

You will be given two integers x and y as input, you have to compute x/y . If x and y are not 32 bit signed integers or if y is zero, exception will occur and you have to report it. Read sample Input/Output to know what to report in case of exceptions.

Sample Input 0:

```
10
3
```

Sample Output 0:

```
3
```

Sample Input 1:

```
10
Hello
```

Sample Output 1:

```
java.util.InputMismatchException
```

Sample Input 2:

```
10
0
```

Sample Output 2:

Line: 21 Col: 2

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0	Input (stdin)	Download
Sample Test case 1	1 10	
Sample Test case 2	2 3	
Sample Test case 3	Your Output (stdout)	
Sample Test case 4	1 3	
	Expected Output	Download
	1 3	